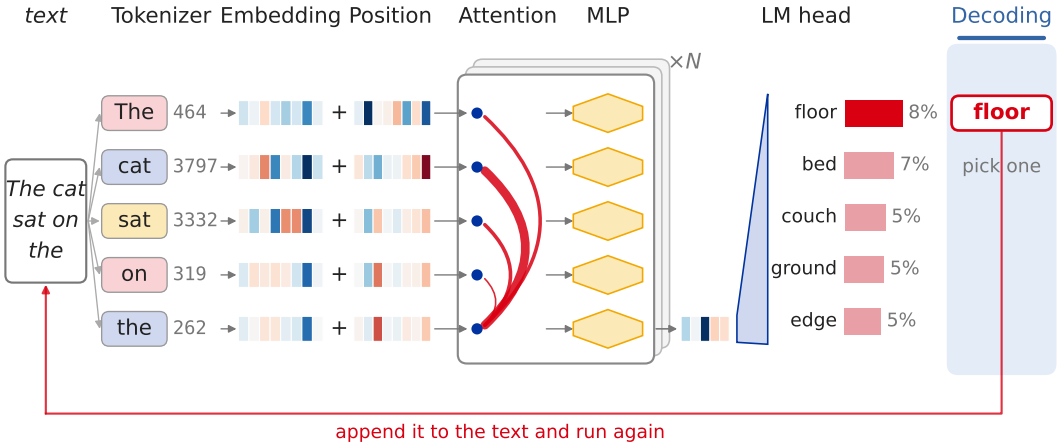
Decoding

From probabilities to text

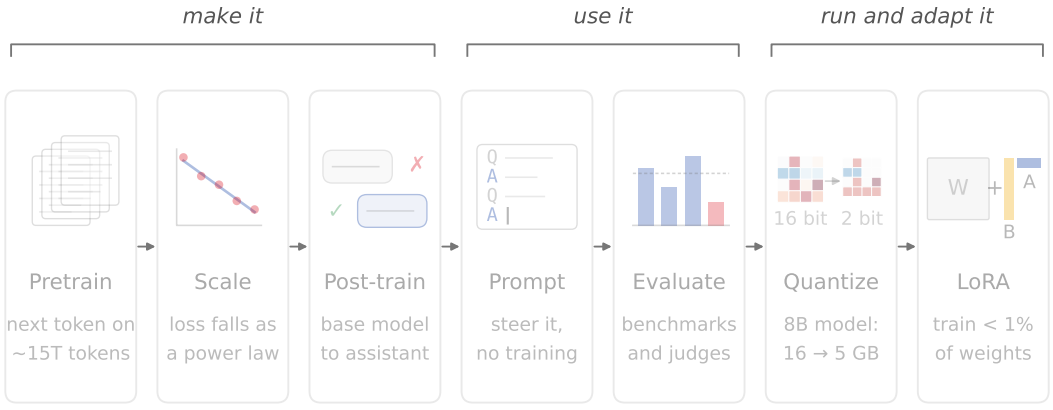
LLM chapter, session 6. Toy numbers unless a slide says otherwise.

Where we are: one forward pass



Today: the last box. The model gives a probability for every token - which one do we write?

Where we are: the life of a model



Later: where those probabilities come from, and what you do with a model once you have one.

The model always writes its single most likely next token.
What does the text look like?

Prompt: "The cat sat on the" - the map's sentence. GPT-2, 50 new tokens.

Commit to an answer.

It loops

The cat sat on the floor, and the cat was still asleep. "I'm sorry, I'm sorry," she said. "I'm sorry. I'm sorry. I'm sorry. I'm sorry. I'm sorry. I'm sorry. I'm

GPT-2's real output, always taking the top token (blue: the prompt, red: from the first repeat on). It starts well - "floor" is the map's 7.6% favourite - and then it is stuck.

So why not just take the top token? That question is today's lecture.

Outline

What the model hands you

Take the most likely

Sample

Constrain the choice

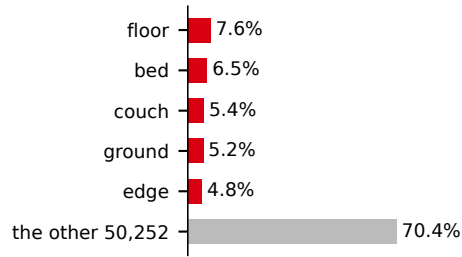
Real systems

What the model hands you

A probability for every token. Writing means choosing one.

One step, one distribution

- L21: generate one token, feed it back, repeat.
- LLM-5: each step ends in logits → softmax → a probability for every one of 50,257 tokens.
- On the map's sentence the favourite, "floor", has **7.6%**. The top five together: 29.6%. The rest: **70.4%**.



The model never chooses a word. It hands over a distribution; **decoding** is the rule that turns it into one token. Real numbers here: GPT-2 on the map's sentence.

When to stop

- **An end token.** The model writes <|endoftext|> (LLM-1's special token). Chat models have an end-of-turn token instead (LLM-9).
- **A length limit.** Stop after N new tokens, whatever happens.
- **Stop strings.** Stop when the text contains, say, "\n\nUser:".

A stop string can be split across two tokens (LLM-1: tokens are not words). So it is checked on the decoded **text**, not on token IDs.

Take the most likely

Greedy, then a smarter greedy.

Greedy

- At every step write the token with the highest probability (the argmax).
- Same input, same output - on one machine (the last section complicates even that).
- Why it loops: once a sentence has appeared, writing it again becomes more likely, and every repeat makes the next one likelier still. Xu et al. (2022) measured this **self-reinforcement**: the more times a sentence is already in the context, the higher its probability of coming next.

Greedy picks the best **next** token, one step at a time. It never asks whether the whole text is likely.

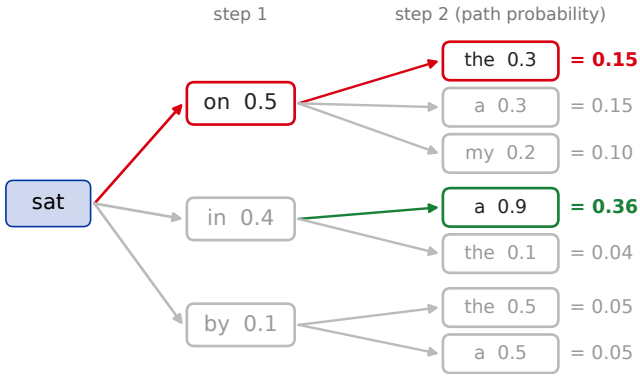
Why we work with log-probabilities

The probability of a whole text is the product of its token probabilities. Toy: 100 tokens, each with probability 0.1:

$$0.1 \times 0.1 \times \cdots \times 0.1 \;=\; 0.1^{100} \;=\; 10^{-100}$$

- The smallest positive number a float32 can hold is about 10^{-45}. In float32 this product is exactly $\mathbf{0}$, and every text scores the same.
- Take logs and the product becomes a sum: $100 \cdot \log 0.1 = -230.3$. No underflow, and adding is cheaper than multiplying.
- Higher (less negative) log-probability = more likely. APIs report `logprobs` per token: a rough confidence signal (calibration, [14]), used to flag shaky answers, to rank several candidate answers, and to look for likely hallucinations (LLM-11).

Greedy is short-sighted

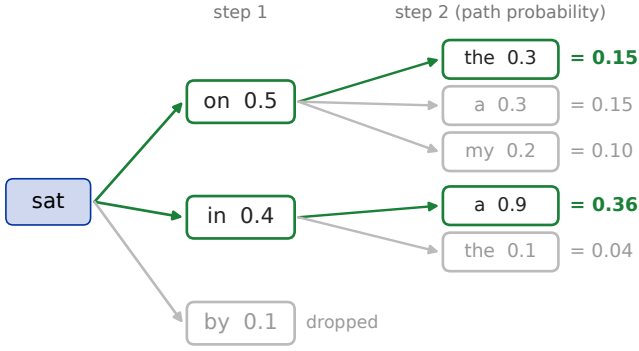


greedy: on, then the
0.5 x 0.3 = 0.15

best: in, then a
0.4 x 0.9 = 0.36

Toy numbers. Greedy takes "on" (0.5 beats 0.4) and ends with a path of probability $0.5 \times 0.3 = 0.15$. The path through "in" was $0.4 \times 0.9 = 0.36$, more than twice as likely - but greedy never looked one step further.

By hand: beam search with $B = 2$



keep the best 2
at every step

winner: in a = 0.36

Keep the B best **partial texts** at every step, scored by their total log-probability. Step 1 keeps "on" and "in" and drops "by". Step 2 extends both and keeps the two best paths: "in a" (0.36) and "on the" (0.15, tied with "on a"). The winner is "in a". With a real vocabulary, each step scores $B \times 50{,}257$ extensions and keeps B.

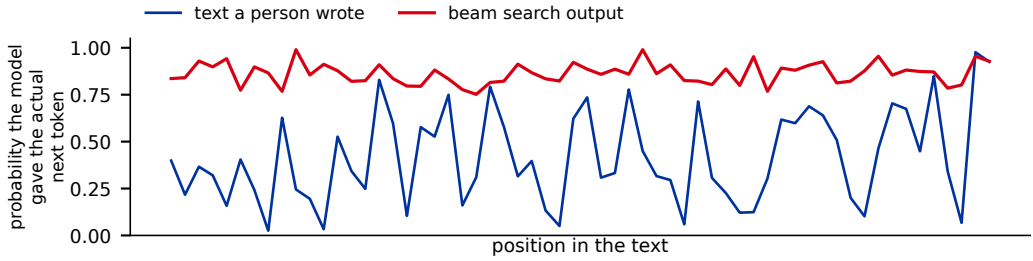
Beam search: why divide by the length

- Every extra token multiplies the probability by a number below 1, so a longer text always has a lower total. Toy: a 3-token ending with 0.5 per token scores $0.5^3 = 0.125$; a 6-token ending with 0.7 per token, each step more convincing, scores $0.7^6 = 0.118$ and loses.
- Plain beam search therefore likes to stop early.
- The usual fix: divide the total log-probability by the length (or by the length raised to a power, one more knob). Now the 6-token ending wins: average log-probability $\log 0.7 = -0.36$ per token against $\log 0.5 = -0.69$.

Beam search finds more probable text than greedy.
Is more probable text better text?

Commit to an answer.

The probability trap



schematic, after Holtzman et al. (2020) - illustrative values

At every position of a text, ask: what probability did the model give the token that actually came next? Holtzman et al. (2020): for text a person wrote it jumps around and is often low; for beam output it stays uniformly high - and that text reads bland and repetitive.

People write **surprising** text, not the most probable text. Maximising probability is the wrong goal for open-ended writing.

Where maximising still wins

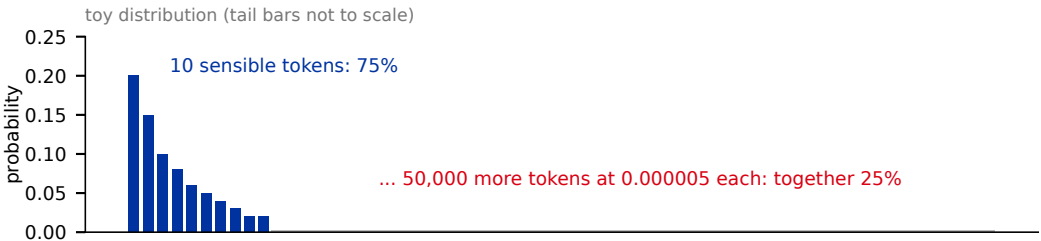
- Tasks with **one right answer** and short outputs: translating a sentence, transcribing speech, answering "what is 17 $\times$ 23?". Here the most probable output is usually the correct one, and translation systems have long used beam search.
- Open-ended text - chat, stories, explanations - is the opposite case: many good answers, and the single most probable one is dull. Chat models sample.

Greedy and beam search answer "what is the most likely text?". The next section answers a different question: "what would a good writer plausibly write?".

Sample

Roll the dice - but with which distribution?

Pure sampling and the long tail



Sampling straight from the model's distribution fixes the loops: the next token is drawn at random, in proportion to its probability. But tens of thousands of unlikely tokens add up. In this toy, 50,000 of them at 0.000005 each make **25%**: one step in four picks something odd, and one odd token can derail every token after it.

Every sampling method below is a way to reshape the distribution, or cut off its tail, before the dice roll.

Temperature

Divide the logits by a number T before the softmax:

$$p_i = \frac{e^{z_i/T}}{\sum_j e^{z_j/T}}$$

- The same operation as **temperature scaling** in calibration ([14], and ch11's lecture 45): there it fixes over-confidence, here it steers how adventurous the text is.
- $T < 1$ widens the gaps between logits: the favourite gets even more probability.
- $T > 1$ shrinks the gaps: the distribution flattens.
- $T \to 0$: all probability on the top token. That is greedy.

By hand: temperature

Three tokens with logits $z = (2,\ 1,\ 0)$:

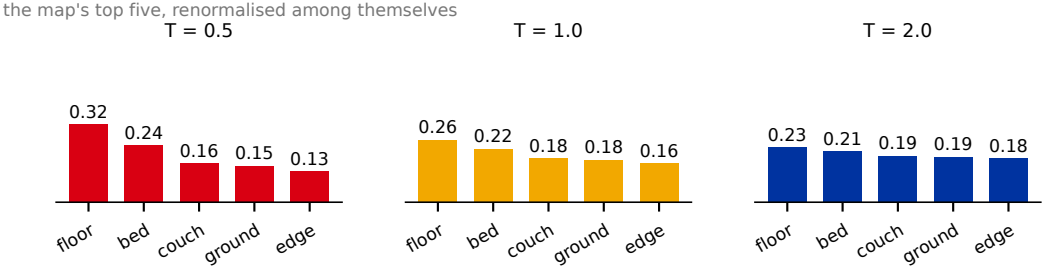
$T = 1$: $\dfrac{(e^2,\ e^1,\ e^0)}{e^2 + e^1 + e^0} = \dfrac{(7.39,\ 2.72,\ 1.00)}{11.11} = (\mathbf{0.67},\ 0.24,\ 0.09)$

$T = 10$: $\dfrac{(e^{0.2},\ e^{0.1},\ e^0)}{e^{0.2} + e^{0.1} + e^0} = \dfrac{(1.22,\ 1.11,\ 1.00)}{3.33} = (0.37,\ 0.33,\ 0.30)$ - nearly uniform

$T \to 0$: the gaps z_i/T grow without bound, so $p \to (1,\ 0,\ 0)$: greedy

Temperature never changes the **order** of the tokens, only how far apart their probabilities are.

Temperature on the map's sentence



The map's real top five, renormalised among themselves at three temperatures. At $T = 0.5$ "floor" pulls ahead; at $T = 2$ the five are almost level. On the full distribution the same happens to all 50,257 tokens: a high temperature hands the long tail more probability.

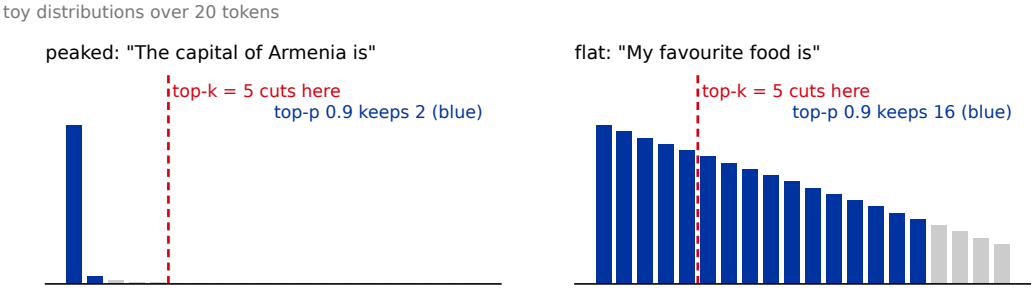
By hand: temperature and the tail

Dividing the logits by T is the same as raising each probability to the power $1/T$ and renormalising. Back to the long-tail toy: 10 sensible tokens with 75%, 50,000 odd ones at 0.000005 each (25%).

- $T = 0.5$: square every probability. The odd tokens' squares are tiny; their share falls from 25% to about **0.001%**.
- $T = 2$: take square roots. One odd token: $\sqrt{0.000005} = 0.0022$; times 50,000 $= 112$. The ten sensible tokens: 2.56 in total. After renormalising, the odd tokens get $112/(112 + 2.56) =$ **98%**.

A high temperature hands the dice to the long tail. That is why temperature is almost always combined with a cut-off - the next three slides - and why the order of the two matters.

Top-k, and why a fixed k fails



Top-k: keep the k most likely tokens, set the rest to 0, renormalise, sample. Toy distributions:

- Peaked: one token has 0.88. Keeping 5 lets four unlikely tokens back in.
- Flat: 20 tokens from 0.08 down to 0.02. Keeping 5 throws away most of the good ones.

The right number of candidates changes at every step. A fixed k cannot follow it.

Top-p (nucleus sampling)

- Top-k's problem was a fixed **count**. Top-p fixes the **amount of probability** instead: keep the smallest set of top tokens whose probabilities add up to at least p; renormalise; sample (Holtzman et al., 2020, the same paper as the probability trap).
- It adapts by itself. Same toys, $p = 0.9$: the peaked one keeps **2** tokens ($0.88 + 0.04 = 0.92$), the flat one keeps **16**.
- The tail of near-zero tokens is cut every time, whatever its size.

Top-k asks "how many?". Top-p asks "how much probability?" - and that is the question that stays sensible from one step to the next.

By hand: top-p on the map's sentence

Real numbers, GPT-2 after "The cat sat on the". Top-p with $p = 0.2$: add the sorted probabilities until the total reaches 0.2.

token	**probability**	**running total**
floor	0.076	0.076
bed	0.065	0.141
couch	0.054	0.195
ground	0.052	**0.247** ≥ 0.2: stop

Keep 4 tokens. Renormalise: floor $0.076/0.247 = 0.31$, bed 0.26, couch 0.22, ground 0.21. Then sample. (A realistic p is 0.9 or more; 0.2 keeps the table short.)

Min-p

Min-p: keep the tokens whose probability is at least min_p times the top token's. Toys, min_p $= 0.1$:

- Peaked: the top is 0.88, so the bar is 0.088 - only **1** token passes.
- Flat: the top is 0.08, so the bar is 0.008 - all **20** pass.

Like top-p it adapts, through a cut-off that scales with the model's confidence.

The gap it targets, its authors argue: at a high temperature the distribution flattens (the tail slide), and top-p's set fills with weak tokens. Min-p's bar is tied to the top token, so it stays strict whenever the model has a clear favourite.

How to read a claim: min-p

- Min-p was an oral presentation at ICLR 2025 (Nguyen et al.), one of the highest-scored papers, with claims of better quality and diversity than top-p.
- A 2025 re-analysis ("Min-p, Max Exaggeration", arXiv 2506.13681) found that it does not beat the standard samplers once they are tuned with the same effort, and found inconsistencies in how the original results were reported.
- Min-p is still a reasonable sampler - llama.cpp turns it on by default. What fell was the claim that it is better.

An oral at a top venue is a starting point, not the last word. Ask: compared with what, tuned how hard, and measured by whom?

The order of the knobs



Hugging Face's order (transformers 5.15). llama.cpp's default chain moves temperature to the end, after the cut-offs.

- Greedy is the special case $T \to 0$, or top-k with $k = 1$.
- Libraries do not agree on the order. Hugging Face applies temperature **before** the cut-offs; llama.cpp's default chain applies it **last**. With top-p or min-p that changes which tokens survive: a high temperature flattens the distribution first, and top-p then keeps more tokens.

"temperature 0.8, top-p 0.95" is not one setting. Check the library.

transformers 5.15 `generate()` source; llama.cpp `--samplers` default: penalties; ...; top_k; ...; top_p; min_p; ...; temperature. Checked 5 Oct 2026.

By hand: the order matters

Toy distribution $(0.5,\ 0.3,\ 0.15,\ 0.05)$, temperature $T = 2$, top-p $= 0.8$:

Temperature first (Hugging Face): square roots, renormalised: $(0.38,\ 0.29,\ 0.21,\ 0.12)$. Top-p 0.8: $0.38 + 0.29 = 0.67$, plus $0.21 = 0.88$ - keep **3** tokens.

Cut first (llama.cpp's default chain): top-p 0.8 on the original: $0.5 + 0.3 = 0.80$ - keep **2** tokens. Temperature then flattens just those two.

Same two settings, different sets of candidates. The third token can only ever be written in the first library.

Repetition penalties: two different things

- **Multiplicative** (Keskar et al., 2019, the CTRL paper; Hugging Face's and llama.cpp's `repetition_penalty`): for every token already in the text, divide its logit by the penalty (multiply it, if the logit is negative). Toy, penalty 1.3: a logit of 2.6 becomes 2.0.
- **Subtractive** (OpenAI's API): subtract a fixed amount from every token that has appeared (presence penalty), plus an amount per appearance (frequency penalty):

$$z_j \leftarrow z_j - c_j \cdot \alpha_{\text{frequency}} - [c_j > 0] \cdot \alpha_{\text{presence}}$$

where c_j counts how often token j has appeared. Toy: a token used 3 times, both penalties 0.5: its logit drops by $1.5 + 0.5 = 2.0$.

Same name, different maths, different scales. Too strong and the model avoids words it genuinely needs to repeat - names, "the".

Seeds

- Sampling draws random numbers, so the same prompt gives different texts on different runs. That is the point of sampling.
- A fixed **seed** replays the same random numbers, so the same text comes out again - on the same machine, with the same software and the same settings.
- Useful for debugging and for comparing two prompts fairly.

"On the same machine" is doing a lot of work in that sentence. The last section shows why.

Settings in practice

- One real set of defaults: llama.cpp ships temperature 0.8, top-k 40, top-p 0.95, min-p 0.05 (`llama-cli` documentation, checked 5 Oct 2026). All four filters are on at once.
- Lower temperature for code, maths and facts, where one answer is right; higher for brainstorming and fiction, where variety is the point.
- Change one knob at a time, and read your library's documentation for the order.

These are habits, not laws. The right setting depends on the model and on the task.

What you see, what to change

what you see	likely cause	try
the text loops	greedy, or a very low temperature	sample; raise the temperature a little; a mild repetition penalty
odd words, sudden nonsense	the long tail: temperature too high, or no cut-off	lower the temperature; add top-p (or top-k, min-p)
bland, the same answer every time	too little randomness	raise the temperature
needed words avoided (names, "the")	repetition penalty too strong	lower it
broken JSON	free sampling	constrain the choice (next section)
different answers at temperature 0	the server's arithmetic (real-systems section)	accept it, or batch-invariant serving

One knob at a time, one symptom at a time.

Constrain the choice

When the output must follow a format. Not a third option: a filter on top of either.

Masking

- At every step, find every token that would make the output break the required format - a schema for JSON (JavaScript Object Notation, the text format programs exchange data in), a list of allowed answers, any grammar.
- Set their logits to $-\infty$, so their probability is 0. Renormalise the rest.
- Then pick as usual: greedy, beam, or sampled with any of the knobs above.

The model is not retrained and does not "know" about the format. The decoder simply never lets it write an illegal token.

By hand: JSON



The schema says {"city": string, "year": integer}, and the output so far ends with "year": - so a number must come next. Only tokens that continue a number are legal; a quote, a brace, null or a word are masked, however much probability the model gave them. The mask covers all 50,000+ tokens at every step, not just the eight drawn.

Tokens are not characters

- LLM-1: one token can hold several characters, like `":` or `"}`, which cover two steps of the grammar at once.
- So the mask cannot look at one character at a time. For every token it checks the token's whole text against the grammar from the current state.
- Doing that for 50,000 tokens at every step would be slow, so libraries precompute, for each state of the grammar, which tokens are legal.

What it buys, and what it does not

- **Guaranteed format.** Tool calls for agents (L44) and JSON for programs to read always parse.
- **Not guaranteed truth.** A valid `"year"` can still be the wrong year.
- **Not guaranteed complete.** Hit the length limit in the middle of an object and the JSON is cut off anyway.

Constrained decoding checks the shape of the answer, never its content.

Real systems

What changes when a server does the decoding.

Streaming: stopping in the middle of a letter



- Chat apps show tokens as they arrive (streaming), not after the whole answer is done.
- LLM-1: in UTF-8 the Armenian letter ayb is two bytes, D5 A1, and a byte-level tokenizer can give each byte its own token.
- A token holding only D5 is not text yet. Show it - or hit the length limit right after it - and the screen gets a broken character. So the app must hold back bytes until a character is complete.

Temperature 0. The same prompt, 1,000 times.
How many different answers?

Commit to an answer.

Temperature 0 is not deterministic

Thinking Machines Lab (He, Sept 2025): Qwen3-235B-A22B-Instruct, 1,000 runs, temperature 0 - **80 different completions**. The most common one appeared 78 times.

- Floating-point addition depends on the order. Toy, in float32: $(10^8 + 1) - 10^8 = 0$, but $(10^8 - 10^8) + 1 = 1$.
- A server runs many users' requests together as one big matrix multiplication: a **batch**. The GPU program for each operation (a **kernel**) splits long sums into chunks, and how it splits depends on the batch size - which changes with the load, second to second.
- So your logits differ in the last digits from run to run. Usually nothing changes; when two tokens are nearly tied, the argmax flips and every later token follows a new path. Flips land at different places in different runs: 80 different texts.

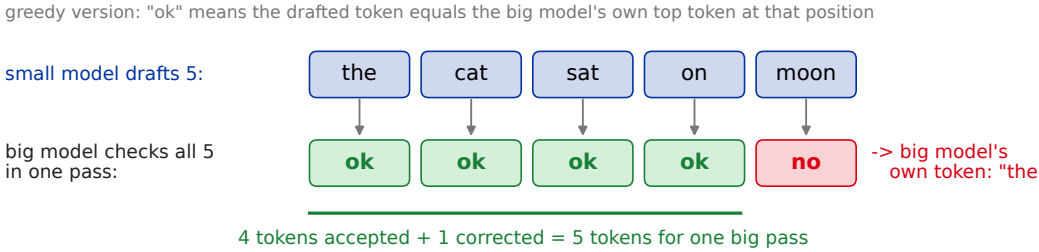
Their fix, batch-invariant kernels, makes all 1,000 identical, at a cost in speed. Greedy decoding is deterministic; the arithmetic underneath it may not be.

Why decoding is slow

- One token per step, and every step is a full forward pass through every layer. 500 tokens of answer = 500 passes, one after another.
- For a single token the GPU has little arithmetic to do, but it must read all of the model's weights from memory each time. Decoding is limited by memory, not by arithmetic (the details are LLM-14).
- Checking several tokens in one pass costs about as much as checking one.

That last point is the whole trick of the next slide.

Speculative decoding



- A small, fast model **drafts** k tokens, one at a time.
- The big model checks all k in **one** pass: the causal mask (LLM-4) gives its prediction at every position at once.
- Keep the drafted tokens up to the first disagreement, take the big model's own token there, and draft again.

Leviathan, Kalman & Matias (2023), ICML; Chen et al. (2023), DeepMind.

By hand: accept or reject

Greedy: accept a drafted token if it equals the big model's own top token. The output is then exactly what the big model would have written alone.

Sampling: accept a drafted token x with probability $\min\big(1,\ p_{\text{big}}(x)/p_{\text{small}}(x)\big)$. Toy: the small model gave x 0.6, the big one 0.3: accept half the time. On a reject, sample from what the big model wanted **more** than the small one did.

Both halves together give tokens with exactly the big model's probabilities - next slide, by hand. **Lossless**: no quality traded for speed. Leviathan et al. report 2-3$\times$ faster generation with T5-XXL.

When the draft is good, most tokens are accepted and each big pass yields several tokens; when it is bad, it costs little more than plain decoding.

By hand: why sampling stays lossless

Three tokens. Small model $p_s = (0.6,\ 0.3,\ 0.1)$, big model $p_b = (0.3,\ 0.5,\ 0.2)$. The small model drafts a token; we accept it with probability $\min(1, p_b/p_s)$.

token	**drafted** p_s	**accept**	**kept**	**shortfall** $\max(0, p_b - p_s)$	**total**
A	0.6	0.5	0.30	0	$0.30 + 0 = \mathbf{0.30}$
B	0.3	1	0.30	0.2	$0.30 + 0.20 = \mathbf{0.50}$
C	0.1	1	0.10	0.1	$0.10 + 0.10 = \mathbf{0.20}$

- Kept: $0.30 + 0.30 + 0.10 = 0.70$, so a reject happens with probability 0.30 (only A is ever rejected: the small model was over-eager about it).
- On a reject, sample from the shortfall $(0,\ 0.2,\ 0.1)$, renormalised: $(0,\ \frac{2}{3},\ \frac{1}{3})$. That adds $0.30 \times (0,\ \frac{2}{3},\ \frac{1}{3}) = (0,\ 0.20,\ 0.10)$.

Totals $(0.30,\ 0.50,\ 0.20)$: exactly the big model. Keep what the small model did not overdo, resample exactly what it underdid. (The name for this trick: rejection sampling.)

Where the draft comes from

- A small model of the same family, with the same tokenizer.
- Extra prediction heads trained on top of the big model itself.
- Copying: in summaries and code edits the next tokens are often already in the prompt.
- Multi-token prediction: models trained to guess several tokens at once (LLM-14).

Speculative decoding changes **how fast** tokens come out, never **which** tokens come out. That is why serving systems can switch it on silently.

Watermarking lives in the sampler

SynthID-Text (Dathathri et al., *Nature*, 2024), used in Google's Gemini:

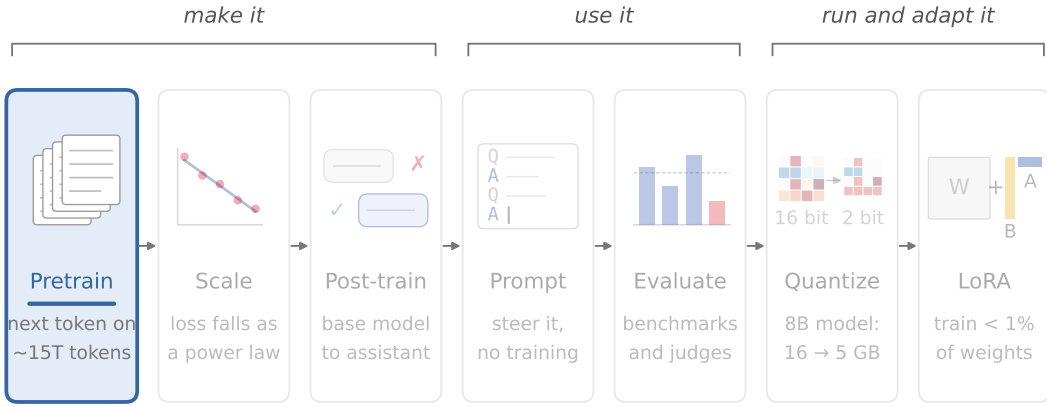
- When several candidate tokens are about equally good, a secret, keyed function decides which one wins. The model itself is untouched; only the sampling step changes.
- Anyone with the key can score a text: if its token choices agree with the keyed function far more often than chance, it is watermarked. Toy: the function prefers half of all candidates, so ordinary text hits a preferred token about 50% of the time; watermarked text, say, 70%. Over a few hundred tokens that gap cannot be luck.
- A reader cannot see anything: every token was a plausible choice anyway.

Google's own documentation: weaker on factual answers (with one right answer there is no freedom to hide a signal) and when a text is thoroughly rewritten or translated.

Recap

- The model hands over a probability for every token; **decoding** picks one. Stop on an end token, a length limit or a stop string.
- **Take the most likely**: greedy (loops, short-sighted), beam search (looks ahead; right for one-right-answer tasks, bland for open-ended text).
- **Sample**: temperature reshapes; top-k, top-p and min-p cut the tail; penalties discourage loops. Libraries differ in the order.
- **Constrain**: masks guarantee a format, not the truth.
- **Real systems**: temperature 0 can still drift; speculative decoding is free speed; a watermark can hide in the sampler.

Next on the map



Next (LLM-7): where these probabilities come from - pretraining.

Sources & further reading

- Holtzman, Buys, Du, Forbes & Choi (2020), "The Curious Case of Neural Text Degeneration", ICLR (the probability trap, top-p).
- Xu et al. (2022), "Learning to Break the Loop", NeurIPS (self-reinforcing repetition).
- Keskar et al. (2019), "CTRL" (the repetition penalty); OpenAI API documentation (presence and frequency penalties).
- Nguyen et al. (2025), "Turning Up the Heat: Min-p Sampling", ICLR; "Min-p, Max Exaggeration" (2025), arXiv 2506.13681.
- He / Thinking Machines Lab (10 Sept 2025), "Defeating Nondeterminism in LLM Inference".
- Leviathan, Kalman & Matias (2023), "Fast Inference from Transformers via Speculative Decoding", ICML; Chen et al. (2023), "Accelerating Large Language Model Decoding with Speculative Sampling".
- Dathathri et al. (2024), "Scalable watermarking for identifying large language model outputs", *Nature*.

Figures: `py_src/decoding_figs.py` (toy numbers; GPT-2's greedy output and the map's top five are the only real ones).